YARG Contributions — Roadmap

Canonical roadmap for both repos in this project (`yarg-song-server` and the `yarg` client fork). Status as of 2026-09-05.

Two tracks run in parallel. The **server track** is the #1 priority and is sequential — each phase depends on the one before it. The **client track** is independent and can absorb spare cycles at any time.

Phase 0 — Foundations ✅

Repos, remotes, mirrors and the format research that everything else is built on.

- [x] `fatalexception/yarg-song-server` and `fatalexception/yarg` created on Vault2 GitLab
- [x] Song-format research spec written (`docs/research/yarg-song-formats.md`)
- [x] Architecture decision recorded (`docs/ADR-001-server-architecture.md`)
- [x] Project instructions mirrored into `CLAUDE.md` so both machines pick them up
- [x] `yarg` imported from upstream server-side: 21 branches, 172 MB of repository and 180 MB of LFS objects, default branch set to `dev`
- [x] Public push mirrors to `github.com/Coffeehedake` configured and verified: `yarg-song-server` (GitLab mirror 10) and `yarg` (mirror 11), both one-way, all branches, divergent refs not kept. First sync 2026-09-05 16:44 ET, both `finished` with no error, and the GitHub side matches GitLab commit-for-commit.

Deliberately deferred: the `yarg` fork is *not* cloned locally, because nothing touches the client until Phase 3 and ~350 MB of Unity source with LFS is not worth carrying before then.

The original reason was different and is now void: the folder used to sit in the Syncthing `dev-projects` mesh, so a clone here replicated to r7 and Vault2 as well. Syncthing was retired on 2026-09-06 and the machines are independent, so a clone now costs disk on one machine only. The conclusion happens to survive the reason dying — but it survives on the weaker of the two arguments, which is worth knowing if the decision is revisited. Clone it when Phase 3 starts:

```
cd "C:\dev\YARG - Open Source Contributions"
git clone https://gitlab.badassium.com/fatalexception/yarg.git yarg
cd yarg
git remote add upstream https://github.com/YARC-Official/YARG.git
git submodule update --init --recursive
git lfs pull
```

Exit criterion: both repos exist, both push to Vault2, both mirror to GitHub, and the format spec is committed. **Met 2026-09-05.**

`Coffeehedake/yarg` is a real GitHub *fork* of `YARC-Official/YARG`, not a plain repository, because that is the only shape GitHub accepts a pull request to upstream from.

Correction — LFS does propagate. An earlier revision of this roadmap claimed GitLab push mirroring drops LFS objects, and that the fork relationship was what made upstream's ~180 MB resolve. That was asserted from priors, not measured, and it is wrong.

The measurement: a throwaway GitLab project (**not** a fork, so no parent LFS storage to borrow from) with `*.png filter=lfs`, one fresh 1 MB object, push-mirrored to a throwaway **non-fork** GitHub repo. GitHub's LFS batch API returned a download action with no error, and the object downloaded from `github-cloud.githubusercontent.com` at exactly 1,048,576 bytes with a SHA-256 matching the OID. GitLab 18.11.3, git-lfs 3.7.1. Both throwaway repos were deleted afterwards.

So no special handling is needed when client work adds one of the five patterns YARG's `.gitattributes` tracks — `*.png`, `*.exr`, `*.jpg`, `*.fbx`, `*.ttf`. Which is just as well: almost any UI work in Phase 3 adds a `.png`.

The caveat that does survive: the mirror force-overwrites and the GitHub side is a fork, so anything done directly on GitHub is clobbered on the next sync. To open an upstream PR, create the branch on GitLab, let it mirror, open the PR from the mirrored branch, and leave it alone on GitHub after that.

Phase 1 — `yargsong`: the Go format library ✅

The server is worthless until Go can read and write what YARG reads and writes. This phase is pure library work with no network surface, which makes it the easiest phase to test exhaustively.

Build order (each step is independently testable):

1. **`song.ini` reader** — ✅ done (`internal/songini`). ~130 recognised keys with their types, and a deliberately lenient reader: UTF-8/UTF-16 BOMs, Latin-1 fallback so accented artist names survive, `[song]` / `[Song]` /no-header variance, trailing junk after the section bracket, values containing `=`, and malformed lines skipped rather than fatal. Two behaviours were

checked against upstream rather than guessed: **duplicate keys — last wins** (upstream stores modifiers by plain dictionary assignment), and **keys and section names are lowercased** before lookup. One thing is still assumed and flagged in the source: the exact whitespace and multiple-`=` handling inside `YARGTextReader.ExtractModifierName`, which has not been read. The writer is step 5 below.

2. **`.sng` reader** — ✅ done (`internal/sng`). Implemented as an `fs.FS`, including synthesised directories, so the folder scanner and the `.sng` scanner share exactly one code path; `fstest.TestFS` enforces that. The mask-origin question the research doc flagged as unconfirmed is now settled from `SngFileStream.cs`: **the XOR index is the byte offset within each contained file, restarting at 0 per file**, not the absolute offset in the `.sng`. Upstream reaches the same result only because it decrypts whole 1 MB buffers and 1 MB is a multiple of the 256-byte table; tracking the real offset is equivalent and survives arbitrary read sizes. Malformed input is rejected with an error rather than a panic, which is tested.

Validated against the reference encoder, 2026-09-05. SngCli v0.3.0 (MIT, from `mdsitton/SngFileFormat` — the tool that defines the format) encoded a song folder we wrote, and our reader read the result: same chart bytes, same SHA-1, same metadata, and chunked streaming matching a whole-file read. The archive is committed at `internal/sng/testdata/reference-sngcli-v0.3.0.sng` so this stays a regression test rather than a thing that was true once. Until this, every `.sng` the reader had seen was produced by an encoder written from the same understanding as the reader — which proves only that the two agree with each other.

Two things the real file taught us that synthetic input could not:

- **Duplicate `song.ini` keys: last wins — confirmed independently.** A probe archive with `name = FIRST VALUE` … `name = SECOND VALUE` round-tripped through SngCli as `SECOND VALUE`, matching what we had concluded from reading `IniModifierCollection.cs`.
- **A file's extension can lie about its container.** SngCli emits audio under a `.mp3` name regardless of source format: our `song.wav` came back as `song.mp3`, byte-identical, RIFF header intact. We classify stems by name, as YARG does, so the scan is unaffected — but anything that later *decodes* audio must sniff the container, and our own writer must not reproduce this. There is a test pinning the observation.
- **Scanner** — ✅ done (`internal/scan`, `internal/catalog`). Walks a folder or a `.sng` through the same `fs.FS`, applies the chart-file priority order, classifies stems (all 14 standard, 5 clean and 4 explicit variants), resolves album art with the `cover` key overriding `album.<ext>`, finds background/video/preview, and emits the v1 catalog schema. Unrecognised files are carried in `assets.other` and every `song.ini` key is preserved in `raw_metadata`, so parse-tuning keys we do not model still survive a repack — losing those would change how a chart plays.

A test packs the same content both ways and asserts the folder and the `.sng` produce the same chart hash, metadata and parts. That is the ADR-001 "one code path" claim being enforced rather than merely intended.

Unknown intensity is `-1`, never `0`, for both an explicit `-1` and an absent key — "the charter rated this trivially easy" and "the chart did not say" must not collapse into the same value. Two `diff_*` keys (`diff_drums_real_ps`, `diff_keys_real_ps`) are deliberately left unmapped rather than guessed; the source says why.

`yarg-song-server scan <path>` walks a library and prints the catalog as JSON, so the parsers can be pointed at a real collection before anything sits behind an HTTP API. 4. **Identity** — ✅ done. `SHA1(chart file bytes)`, matching YARG's `HashWrapper`, plus a `package_hash` of our own (SHA-256 over the sorted `name:sha256` pairs) for same-chart-different-audio cases. Note the folder and `.sng` forms of one song have the same chart hash but *different* package hashes, because the folder carries `song.ini` as a file and the `.sng` carries it in the header — that is correct, and the test asserts it rather than papering over it. 5. **`.sng` writer** — ✅ done (`internal/sng/write.go`, `scan.PackDir`, `yarg-song-server pack <folder> <out.sng>`). Validated three ways, weakest first:

- **Our own round-trip** — proves only that our reader and writer agree.
- **The reference decoder.** `SngCli decode` extracted our archive with zero errors, and `notes.chart`, `album.png` and the audio all came back byte-identical to the source folder. Our archive of the same song is the same size as SngCli's, 8,806 bytes.
- **The client.** YARG v0.15.0 scanned 15 archives written by us and accepted **14**, reading every title correctly out of our metadata section — including Latin-1, UTF-16LE, a duplicate key resolving to `SECOND`, and an unknown key surviving the repack.

The one rejection was `No notes found`, and a **control settles it**: SngCli's own archive of the same source folder, scanned alongside ours in the same pass, was rejected with the identical error. The fixture's hand-written chart has no note events. As far as the client is concerned our writer is indistinguishable from the reference encoder.

Deliberate differences from the reference encoder: we do **not** rename audio to `.mp3` (SngCli does this regardless of the source container), and we refuse `song.ini` as a contained file rather than letting it disagree with the metadata section. Filenames are lowercased and collisions after lowercasing are refused, metadata keys are emitted lowercase because YARG matches `.sng` keys against a lowercase table without normalising them, and the header size is asserted against what was actually written — a mismatch there would silently corrupt every file offset. 6. **Chart preparsers** — ✅ done (`internal/chart`). Determines which parts and difficulties a chart actually contains, for both `notes.mid` and `notes.chart`. No timing, no sustains, no HOPO inference — the browse UI needs an instrument grid, and the client already has YARG.Core for everything else.

Built from documentation this time, not from source: TheNathannator's Guitar Game Chart Formats, the RBN/C3 documentation, FireFox2000000's `.chart` spec and the Elite Drums spec. The study is in `docs/research/chart-preparsing.md` with a citation per claim.

The governing rule is never to range-test a difficulty block. Every instrument family has non-playable numbers inside its own blocks, and a range test turns each into a phantom difficulty. Sixteen tests exist mostly to pin the traps the documentation names:

- force-strum and HOPO markers sit inside the block and are not notes;
- five-fret note 59 is a left-hand *animation* note unless an `ENHANCED_OPENS` event says otherwise — counting it unconditionally invents an Easy difficulty on a large share of Rock Band-derived charts;
- every Pro Keys track is *required* to carry a range-shift marker, so a test not strictly inside 48–72 reports even empty tracks as present;
- the Pro Keys animation tracks use an identical note range and differ only by name, so track names are matched whole-string, never as substrings;
- Elite Drums' modifier octave carries a disco-flip marker that exists for downcharting and can sit on a difficulty with no gems;
- a lone `HARM1` is the lead line, not a harmony arrangement.

Drum type is a heuristic because it has to be — 4-lane, Pro and 5-lane share one track — with `song.ini`'s `pro_drums` / `five_lane_drums` overriding, and both set at once recorded as the documented invalid state rather than silently resolved. Elite Drums downcharting is honoured: a song with only `PART ELITE_DRUMS` reports 4-lane, Pro and 5-lane as `derived`, because the client shows them.

The SMF reader is hand-rolled rather than a library, deliberately. Charts violate the MIDI spec in two documented ways — running status not reset after SysEx or meta events, and `0xFF` bytes inside SysEx — and a strict parser rejects files YARG plays fine.

Cross-checked against the oracle: the corpus case whose `notes.mid` YARG rejected with "No notes found" now reports zero parts from our preparser too. Same conclusion, independent route.

Exit criterion: a CLI that scans a real song library, emits a catalog, repacks to `.sng`, and YARG scans the repacked output with identical metadata and an identical hash. **Met 2026-09-05.**

`yarg-song-server scan <path>` and `yarg-song-server pack <folder> <out.sng>` do the first two. For the third: `SngCli` decoded our archives with every file byte-identical to the source, and YARG accepted 14 of the 15 archives we wrote — the one rejection being a fixture whose chart has no note events, proven by a control in which SngCli's own archive of the same folder was rejected identically.

And the parts we report now agree with the client's own verdict: on the corpus (22 cases then, 23 since archive ingest landed), **every song YARG rejects is one this scanner independently flags**, by three routes — no parts detected, `no_audio`, and `ultrastar_no_title`.

Explicitly out of scope, permanently: CON/mogg decryption, `songcache.bin` generation, `.milo_xbox` / `.png_xbox` decoding, `.yarground` inspection, full MIDI chart semantics.

Phase 2 — The server, and a sync client that needs no client changes

Two deliverables. The sync client is what makes the server *useful* before any client fork work exists, and it is the proof that the server is correct.

2a — `yarg-song-server`

- [x] **HTTP API** — `internal/httpapi`, documented in `docs/API.md`. Browse and search over the 12 attributes YARG itself sorts by, `GET /song/{chart_hash}.sng`, and a bulk `POST /api/v1/have` that takes the hashes a client holds and answers what it is missing.
- [x] **The index and the store** — `internal/library`, in memory and rebuilt at start; `internal/packcache` packs a loose folder to `.sng` on demand and caches it by package hash. Both decisions and their costs are in `docs/ADR-002-v1-store.md`.
- [x] **Sort parity with the client** — `internal/sortkey` reproduces YARG.Core's `SortString` (rich-text stripping, diacritic folding, whitespace collapse, article removal, character grouping, UTF-16 ordinal comparison) and `internal/library` orders by upstream's own comparer chains. Without this a browse list is internally consistent and unlike anything the player sees in the game.
- [x] **Ingest: loose folders, `.sng`, and `.zip` / `.7z` of a loose folder** — done 2026-09-06. `internal/scan/source.go`. `.zip` uses the standard library; `.7z` took a dependency, recorded with its measured cost in `ADR-003` (+1.62 MB, 3 → 71 compiled packages, no cloud stack despite what `go list -m all` implies).

  No adapter was needed and no scanning logic was duplicated. `archive/zip`'s Reader and sevenzip's Reader both already implement `fs.FS`, and the scanner has taken an `fs.FS` since Phase 1, so an archived song is read by exactly the code that reads a loose folder. `PackDir` became a one-line wrapper over a new `PackFS`.

  The property that matters: **a song packs to the same bytes whether it is loose, zipped or 7z'd**, so restructuring a library does not make every client re-download every song. `TestAllThreeContainersProduceTheSameArchive` asserts it directly, and the corpus now carries a zipped case (`23-zipped`) so the oracle exercises it end to end.

  RB packages are refused with a reason, not ignored — `.con`, `_rb3con`, `.pkg`, `.xex`, matched by SUFFIX because `_rb3con` files usually have no extension at all. An archive holding several songs raises `ErrTooManySongs` rather than publishing one and silently dropping the rest.

  Hardened against hostile archives, by probing rather than by reasoning — a throwaway test printed what actually happens for traversal entries, corrupt files, empty archives and odd separators, and only then were assertions written. Path traversal is dropped by `fs.ValidPath`

before it can reach a served `.sng`; a corrupt archive is reported and the walk continues.

The probe found a real defect: a zip written with **backslash** separators (`Song\song.ini`, as some Windows tools produce) surfaced a directory with nothing readable under it, resolved to `ErrNoChart`, and was **silently ignored** — a legitimate song vanishing from a library with no message, the same failure this project had already rejected for `_rb3con`. Such an archive now reports `ErrUnreadableArchive`. `internal/scan/hostile_test.go`; the shapes measured are tabulated in `TEST-CORPUS` and the reasoning is in `ADR-003`. - [x] **Config file + sane defaults** — `internal/config`. `key = value` with `#` comments, the same names as the flags, precedence defaults < file < flags actually typed, and `--write-config` to print a commented example. An unknown setting is an error, not a warning. A file NAMED on the command line and missing is fatal; the conventional `./yarg-song-server.conf` simply not existing is the normal first run and is silent. - [x] **CI** — `.gitlab-ci.yml`. `gofmt`, `go vet`, the suite, the suite again with `-race`, all six release targets, and an assertion that the Dockerfile's Go is not older than `go.mod` requires. See below for what this replaced. - [x] **A bound and an eviction policy for the pack cache** — done. `pack_cache_max`, defaulting to **2 GiB** rather than to unbounded, with LRU eviction. The number that decided the default was measured on the vault2 deployment: 225,406 bytes of loose library produced 229,515 bytes of cache across 22 archives, **a ratio of 1.02** — so an unbounded cache over a loose library eventually needs a second copy of that library on the data disk. On the Pi target, with the library on external storage and `-data` on the SD card, that fills the card. Eviction costs a re-pack and never data, because the archive is rebuilt byte-identically and its package hash comes from the content. **That last sentence was written on 2026-09-05 and was untrue until 2026-09-06** - packing drew a random mask, so a re-pack produced a different archive; see the determinism entry below. Two smaller leaks went with it: the per-key lock map (now 256 fixed shards, bounded by construction) and orphaned `.partial` files from a crash mid-pack (now swept at start). `internal/packcache` had no tests at all; it now has six, each red-proofed. - [x] **The multi-arch container image** — `container-image` in `.gitlab-ci.yml`, pushing `linux/amd64` + `linux/arm64` to `registry.badassium.com/fatalexception/yarg-song-server`.

And it needs no emulation, which is the part worth remembering. The earlier plan here was to install qemu/binfmt on Vault2. That plan was wrong. Go cross-compiles natively, and the Dockerfile already pins its build stage with `FROM --platform=$BUILDPLATFORM`, so the toolchain runs at the host's own architecture and Go emits the arm64 binary itself. Emulation only enters if the build stage is pulled *for* the target platform, which that directive prevents. **Nothing on the Vault2 host was changed, and nothing should be.**

That correction came from the ShopStack session, who also established that `juniper-pi-deploy` — the "Pi framework" this was going to be cloned from — builds bootable SD-card images and publishes no container images at all. Same word, different problem.

Two things the job does that are not obvious:

- **It creates a `docker-container` buildx builder.** The default builder uses the `docker` driver, which cannot build more than one platform and cannot produce a manifest list at all. Without this step buildx fails with a message about the driver rather than anything that mentions multi-arch.
- **It verifies the result rather than trusting the exit code.** `ci/verify-multiarch.sh` asserts the manifest covers both platforms *and* reads the ELF machine type of the binary inside the arm64 image. An amd64 binary under an arm64 manifest entry passes every check that only reads exit codes, and then fails to start on the Pi. The check is structural rather than execution-based on purpose: running the arm64 binary in CI would need the emulation this project deliberately does not have.

That structural check was doing its job, but it is not the same as an execution, and this document said so for weeks. On 2026-09-06 the gap was closed on real hardware instead of in CI — a Raspberry Pi 4 ran both the arm64 binary and the published arm64 image. CI's job is unchanged: catch a mislabelled manifest early. Proving it *runs* stays a hardware exercise.

What CI replaced. This project had no `.gitlab-ci.yml`, so GitLab fell through to Auto DevOps. Pipeline #2216 — the only pipeline this project has ever run — failed three jobs with exit 127 trying to execute `/build/build.sh`, `/bin/herokuish` and `lsif-go`. Those are container-executor assumptions, and the runner is a **shell** executor on Vault2, which has none of them. A red pipeline nobody believes is worse than no pipeline, so the CI here brings its own pinned Go toolchain (SHA-256 verified before use, cached between runs) rather than depending on whatever happens to be installed on the host.

Measured end to end on 2026-09-05, against the 22-case corpus: the server indexed all 22 in 15 ms, `POST /have` from an empty client reported 22 missing, all 22 downloaded through `GET /song/{hash}.sng`, and re-scanning the downloaded archives gave 22 songs and 0 failures.

Then the oracle, which is the only test that means anything here: **YARG v0.15.0 scanned the 22 archives the server produced and accepted 20.** The two it refused were the two corpus cases built to be refused, and our own scanner flags both independently — `No notes found` against a preparser reporting no parts at all, and `No audio accompanying the chart file` against our `no_audio` issue. The standard holds on the served archives as well as on the folders.

One thing that run turned up, worth knowing before Phase 2b. Repacking a loose UltraStar folder whose `notes.txt` has no `#TITLE` makes YARG *accept* a song it refuses as a folder. As a folder the title has to come from the chart, and a missing `#TITLE` is fatal — "Name metadata not provided". Packed, the name lives in the `.sng` metadata section, which the packer fills from `song.ini`, so the song has a title and plays. Our scanner agrees with the client in both forms (`ultrastar_no_title` on the folder, no issue on the archive), so nothing here is broken — but it means "the server serves exactly what the folder was" is not quite true for this one format, and a sync client will show a song the player could not previously play.

2b — sync client

A small companion binary that pulls the server's library into an ordinary local songs folder. Unmodified YARG then just sees files. This ships real value in days, with zero risk to the client, and exercises the whole server end-to-end.

- [x] `yarg-sync` — `internal/syncclient` and `cmd/yarg-sync`, documented in `docs/SYNC-CLIENT.md`. Writes `<chart_hash>.sng` and nothing else, verifies every download by re-deriving identity from the bytes it received, resolves a shared chart hash deterministically, and leaves everything it did not write strictly alone — including under `-prune`, which is off by default. Built for all six release platforms.
- [x] **End-to-end coverage** — `internal/e2e` runs the real `httpapi.Server`, `library.Build` and `packcache` against the real client. Stubs prove one side's logic; only this catches the two sides disagreeing about the wire. All five tests were red-proofed by breaking the code they cover.
- [x] **Run it against the oracle** — done 2026-09-05. YARG v0.15.0 pointed at a folder `yarg-sync` filled: **20 of 22 accepted**, the two refusals being the two our scanner flags. "Unmodified YARG just sees files" is now a measured result rather than a claim. The run found the fifth oracle finding and two real defects — a `notes.mid` with no note tracks was not flagged at all, and `PreparseUltraStar` skipped note derivation whenever `#TITLE` was missing, so a chart reported zero parts for a song that plays. Both fixed and red-proofed; write-up in `docs/TEST-CORPUS.md`.
- [x] **Run the server as a container, off the development machine** — done 2026-09-05 on vault2. The published image had never actually been executed before this; CI only ever checked the image config's architecture and the binary's ELF machine type, which are checks on bytes. It now runs: 8.6 MB distroless, indexed 22 songs in 7 ms, `version=827e0fe`. `yarg-sync` from ENG-1 over Tailscale pulled 22/22 in 341 ms — recorded at the time as "byte-identical to the localhost run", which was a **byte-count** comparison and not a per-file one; the files were in fact not identical, for the reason below — and a second sync transferred nothing. **Unmodified YARG on the result: 20 accepted, 2 refused** — the same two, both flagged by our scanner. Write-up in `docs/DEPLOY-VAULT2.md`.
- [x] **Execute arm64 on real ARM hardware** — done 2026-09-06 on a **Raspberry Pi 4 Model B Rev 1.5**, `aarch64`, Debian 13 trixie. Both halves ran:
  - the cross-compiled `linux/arm64` **binary**, confirmed by the Pi's own `file` as `ELF 64-bit LSB executable, ARM aarch64, statically linked`, indexing 22 songs in 24 ms;
  - the published multi-arch **container image**, `arch=arm64`, image id `3c5fdd28…`, `version=1ae2219`, indexing 22 songs in 25 ms under docker.io 26.1.5.

  Until this run, "portable to Raspberry Pi" rested entirely on ELF machine type and image config — checks on bytes, never an execution. It is now a measured result.

  And it extended the determinism result. Archives served by the arm64 server, both as a bare binary and as the container, are byte-for-byte identical to all five earlier x86-64 sync sets. Seven independent syncs, two operating systems, two CPU architectures, 154 archives, one set

of bytes. The mask derivation is architecture-independent, which it had to be and which nothing had tested. Write-up in `docs/TEST-CORPUS.md`.

Caveats stated rather than buried: the board was a **Pi 4, not the 3B+** this project had been waiting on (that board is dead — steady red PWR, no ACT activity, with three independently written cards). The Pi was a **one-shot machine, wiped afterwards**, so this is "arm64 ran on this date", not a maintained deployment. And the image reached it by `docker save` on vault2 and `docker load` on the Pi rather than a direct registry pull, to avoid putting a registry credential on a throwaway host — the image id is identical either way, so what ran is the published image; only its transport differed.

The one-shot shape is deliberate, not a shortcut. The Pi is a *verification target*, not a development or deployment one: development happens on Docker, and a Pi is picked up periodically to confirm the arm64 build still runs on real ARM silicon. So there is no maintained Pi to keep current, and no Pi-shaped work waiting on hardware — each arm64 claim is dated, and re-measured rather than inherited. - [x] **The exit criterion, with a second client** — done 2026-09-06. YARG was installed on r7-desktop by copying ENG-1's portable install, and both machines synced from one server and were scanned by unmodified YARG: **20 accepted, 2 refused on each, the same two songs for the same two reasons.** - [x] **Deterministic packing** — found by that run and only findable by it. The two machines received 22 archives each, 229,515 bytes each, 0 failures each — and **16 of the 22 files differed**. `sng.Write` drew its header mask from `crypto/rand` on every call, so `PackDir` was not a function of its input, and 16 was exactly the number the bounded cache had evicted and re-packed between the two syncs. That broke the strong `ETag`, broke a `Range` resume spanning an eviction, and made "two machines syncing one server get the same bytes" false wherever this repo asserted it. The mask is now derived from the package hash (`sng.MaskKeyFor`); it is stored in plaintext in the header regardless, so deriving it gives nothing away. **22/22 identical across a full cache wipe and across the two machines.** Two of the things that hid it are worth more than the fix: a same-song-twice test that was always a cache *hit* and therefore passed whatever the packer did, and `TestWriteUsesAFreshMask`, which actively *required* the non-determinism. Write-up in `docs/TEST-CORPUS.md`. - [x] **Re-measure the containerised chain against the fix** — done 2026-09-06. vault2 re-pulled onto `c623dae` with its pack cache wiped, then five independent syncs compared file by file: ENG-1 and r7 against the working-tree server, ENG-1 against that server after a full cache wipe, and both machines against the container. **110 archives, all one set of bytes.** Two servers built for different operating systems by different toolchains agree, so **the mask derivation is not platform-dependent** — which it had to be, and which nothing had shown. Unmodified YARG on the container's output: 20 accepted, 2 refused, the same two. - [x] **Deploy the hardening and measure it on the real server** — done 2026-09-06. vault2 re-pulled onto `077f36e`, pack cache wiped, all 23 songs re-packed by the new binary: **byte-for-byte identical to the `423902b` cache**, and the 23 files a `yarg-sync` then received hashed to the same set as the 23 packs on the server. Every earlier determinism result compared machines or architectures at a *fixed* commit; this one holds **across a code change**, which is the case an `ETag` and a `Range` resume actually meet, since a server is upgraded far more often than it changes CPU.

And the defect was reproduced on the deployment before being declared fixed. A probe archive with backslash separators was dropped into the host library: the server indexed 23 songs with `problems=1`, naming the file and saying how to fix it, in the log and in `/api/v1/library` both. The previous image reported `problems=0` and said nothing. Probe removed, library restored to 23 clean cases. See `docs/DEPLOY-VAULT2.md`. - [x] **Measure what this costs at real library sizes** — done 2026-09-06, `cmd/mkscale` and `docs/SCALE.md`. Every number in this repository until now came from 23 songs and 238 KB. Now: **index is linear**, 0.594 / 0.583 / 0.569 ms per song at 1,000 / 10,000 / 31,109 songs, and memory fits **13 MB + 6.5 KB per song** across the whole range.

Those constants extrapolate, and one of them is a constraint on goal #1: **a 100,000-song library needs ~665 MB resident**, so a 1 GB Pi 3B+ could not hold it while serving from it and a 4 GB Pi could. The catalog is in memory by design (ADR-002); this is the number that says where that design runs out.

On the byte axis — 200 songs of 5 MB audio, 1.07 GB — **resident memory stayed at 16.5 MB while serving 1.17 GB**, so packs stream rather than being assembled in memory, which the code was written to do and nothing had measured. With the cache bound set to 100 MB against that 1.17 GB library, **all 200 songs were still delivered and the cache never exceeded 102 MB** — the `pack_cache_max` property re-measured under 11x pressure rather than trusted.

The library is generated, not downloaded: community charts carry copyrighted audio, and a corpus that cannot be rebuilt on another machine is not a corpus. **What this does not close:** the songs are uniform, so a real library's *variety* is untested, and the oracle has never run at scale — that is the measurement most likely to find a YARG rejection category the scanner misses, and it needs real charts. - [x] **The oracle, against REAL charts and as a script** — done 2026-09-07. `scripts/oracle.ps1` replaces a documented hand procedure: it repoints YARG's `SongFolders`, wipes the cache, launches, waits, compares both verdicts, and restores `settings.json` in a `finally` so a crash still puts the operator's YARG back. It waits on the **song cache** rather than `badsongs.txt`, because a library YARG is happy with writes no `badsongs.txt` at all, and it requires that file to be newer than the launch so last run's verdict can never be read as this one's.

Run against **128 real community songs** (1.16 GB, supplied by Jay): YARG refused 0, we flagged 0. **A weak positive, and recorded as one** — both sides agreeing means there were no disagreements available to find, and a curated pack people actually play is close to the least informative sample for this test. What it does establish is **zero false positives on real charts**, which nothing had tested, and that `ErrTooManySongs` is right about real pack `.zip`s rather than only synthetic ones. - [x] **Un-skip upstream's own scanner tests** — done 2026-09-07. `YARG.Core.UnitTests`' `FullScan` and `QuickScan` skip unless `YARG_TEST_SONG_DIRS` names a song directory. Pointed at the same 128 songs, both pass and YARG.Core writes no `badsongs.txt` — a second oracle that runs **in-process in seconds**, needing only the .NET

SDK rather than the game. Upstream's own comment says the only fail condition is an unhandled exception, so it is a crash test over real input, not a verdict test; `docs/TEST-CORPUS.md` says so rather than letting it read as more.

Still uncovered: all 128 songs are `.mid`, so real-world `.chart` coverage is zero — and `.chart` is the format whose early-return bug this project already found once. - [x] **Measure what several clients at once do — and fix what it found** — done 2026-09-07. Every measurement before this was serial, while the whole point of the project is a server on a LAN with more than one client. The probe found a **real defect**: the song handler asked the pack cache for a *path* and opened it as a second step, so an evicting goroutine could remove the archive in between and the server answered **404 "this song is no longer where the index says it is; rescan"** for a song that was present. Measured at 64 clients over a 40-song library bounded to two archives: **2, 1 and 0 spurious 404s across three runs of 2,560 requests** — rare enough never to be met by hand, constant on a busy server, and it points the operator at a library that is fine.

`packcache.Open` now returns an **open file**, so there is never a moment where the caller holds a name but not a handle. Two properties held under the same pressure: the thundering herd collapses (64 concurrent requests for one cold song → one pack, one archive, 64 identical responses), and **eviction costs a re-pack and never data** — 7,680 requests against a cache bounded to a *single* archive returned byte-identical archives, zero mismatches. One promise was corrected: `pack_cache_max` is enforced after each insert, so it is a **high-water target rather than a hard cap**, overshooting by one to three archives as concurrency rises. Details in `ADR-002`; the instrument mistakes are in `SCALE.md`.

Resolved, and worth remembering for the response rather than the event: on 2026-09-05 a Defender machine-learning verdict (`Trojan:Win32/Bearfoos.A!ml`) quarantined `cmd/yarg-sync` builds for about a minute, then stopped reproducing within the hour with nothing changed on the machine — verified across three forced relinks and three distinct binary hashes, zero new detections, same signature version, no exclusion added. Acting on it immediately would have left a permanent Defender exclusion behind for a problem that had already evaporated. If it returns, re-measure before doing anything structural; the escalation is Microsoft's false-positive form and code signing, never an exclusion. Details in `docs/SYNC-CLIENT.md`.

Exit criterion: a Pi on the LAN serves a shared library; two machines running stock YARG both see the same songs without either one being modified.

Phase 3 — Native remote song source in the client fork

Only now does the `yarg/` fork get touched. This is the upstream-facing work.

- Add an HTTP song source alongside local folders — browse, stream, cache.

- Touches YARG's song cache, so it must be designed with upstream in mind rather than bolted on.
- PRs target `dev`. Never `master`.

Decided 2026-09-06: build it in the fork now, and approach upstream in parallel. Their answer shapes this work; it does not block it. That is a deliberate choice rather than impatience — their own contributing guide invites experimentation on undecided features and says large progress can promote a tier, so a working implementation is a better opening than a proposal.

Reconnaissance done 2026-09-06, recorded in `UPSTREAM.md` with the draft post:

- **Their `CONTRIBUTING.md` sorts every feature into six tiers, and the tier decides whether a PR is even read.** A remote song source matches none of the published examples, so *which tier* is the first question — and asking it on Discord before building is what their guide explicitly tells contributors to do.
- **Nothing upstream proposes this.** Searched their issues: the nearest is **#860**, a built-in web server for search and queueing from a phone — a *control plane*, not a content source, and worth not conflating with this. `#1030 "user-supplied song sources"` is about source **icons**.
- **Two of our three permanent non-goals are things upstream has independently ruled out.** CON Decryption is in their Out of Scope list, verbatim: "*Do NOT PR these features. Your PR will immediately be denied.*" We are not asking them for anything they have already refused, and saying so is worth a sentence in the opening post.
- **The ask is smaller than it sounds.** The server already hands out plain `.sng` that unmodified YARG reads, so this is not "support our protocol to play our songs" — it is "fetch from a URL instead of needing a separate sync tool". The seam underneath is that `SongEntry` is abstract but `ActualLocation`, `SortBasedLocation` and `GetLastWriteTime()` all assume a local path; *a song entry whose bytes are not on disk* is a general capability rather than a feature about our server, and may be an easier thing for upstream to want.

A second payoff, which strengthens the case upstream. The same client-to-server channel would carry a **queue**, which is what upstream's open issue #860 has been asking for since August 2024 — search and queue from a phone while YARG is running. That request is unbuilt because nothing can reach the running game; a remote song source is the thing that could. See "Party mode" under Phase 4.

Exit criterion: the fork can browse and play from a server without a sync step, and a discussion thread exists upstream.

Phase 4 — Modular features and the server config menu

The point at which the server stops being one feature and becomes a platform.

- Feature registry: each capability is opt-in and independently enable-able.

- Config UI in the server app, so a user turns on only what they want.
- Candidate modules: multi-user libraries and permissions, playlists/setlists shared across clients, scores and leaderboards, library health reporting.

Party mode: a web UI for search and queueing from a phone

Upstream has an **open feature request for exactly this** — #860, filed August 2024, still open and unlabelled, with a comment pointing at a second Discord proposal that adds up/down votes on the queue. So there is demand, and nobody has built it.

Half of it is ours already and half of it is not, and the split is worth being precise about rather than filing this as "just add a UI":

- **The browse-and-search half needs no new capability.** `/api/v1/songs` already does free text across name, artist, album, genre, subgenre, charter, source and playlist, with twelve sort attributes, ordering and paging, and it answers a 10,000-song catalog in ~140 ms. A phone-friendly page over that API is presentation work on an API that exists.
- **The queueing half needs the game.** The request says "*whilst YARG is running*", and that is the whole difficulty: this server has no channel into a running client, and polling a folder is not one. A queue can live here as server-side state, but something in the game has to read it.

That second half is the same work as Phase 3, seen from the other end — and that is the useful realisation, not a coincidence to note in passing. A client that can talk to a server for songs can read a queue from the same server over the same channel. It means the remote source has two payoffs rather than one, and it means this project would be answering an open request of upstream's rather than proposing something novel.

Order follows from that: **build the web UI when the server is otherwise idle** — it is useful on its own for picking songs from the couch, and it needs nothing from anyone — but do not promise the queue until Phase 3 has a channel to carry it.

The browse half shipped 2026-09-07. `GET /` serves a phone-friendly page: search, the twelve sort attributes read from `/api/v1/library` so the page cannot drift from the server, and a per-song panel with metadata, parts, the scanner's issues and a download link. Embedded in the binary — no CDN, no build step — because a Pi at a party has no internet to fetch a stylesheet from. On by default (`browse_ui`); it exposes nothing the API did not already.

Registered as `GET /{$}`, and that detail is load-bearing: a bare `GET /` in Go's ServeMux is a catch-all that would answer every unmatched path with the page and a 200, turning every documented 404 into HTML a sync client would try to parse as a `.sng`. Red-proofed.

Still no queue, deliberately, and no UI hinting at one.

Phase 5 — LLM chart generation

The long-term goal, and the phase most likely to move. It depends on every phase above working.

- **Start from the tools that already exist.** Help:Charting names two, both free, that do the first half of this: **Ultimate Vocal Remover (UVR)** separates a mix into bass, drums, vocals and other stems, and Spotify's **Basic Pitch** generates pitched MIDI from a vocal stem — a baseline vocals chart to work from. The novel part of this phase is the instrument charting and the difficulty reduction, not stem separation, and building either from scratch would be waste.
- Upload any song; auto-generate instrument and vocal parts across all difficulty levels.
- Produce a complete, working, **editable** package — generated charts are a starting point, not a final answer.
- **Distribution constraint, non-negotiable:** the chart/vocal tracks must be packageable and distributable *separately from the audio*, so charts can be shared without the music.
- Note what YARN's guidelines establish about this: **visual synchronisation is itself a derivative work**, which is why they refuse no-derivatives licences. A generated chart is a derivative of the song it was generated from, not a neutral companion to it — so separating chart from audio reduces the problem, and does not by itself dissolve it.
- Runs against the existing GPU arbitration on the home estate rather than a new stack.

Client track (independent of the server line)

Can be picked up at any time; does not block the server.

- **Controller compatibility** — official Rock Band and Guitar Hero instruments first. Upstream handles this through PlasticBand-Unity, so fixes may belong there rather than in YARG.
- **Graphics** — general rendering and visual improvements.
- Both are ordinary upstream contributions: fork, branch off `dev`, PR to `dev`.

Blockers

None.

Toolchain and credentials, as measured

The Coffeehedake GitHub PAT was rotated on 2026-09-05 and verified (`login=Coffeehedake`, scopes `repo`, `workflow`, `project`, `write:packages`, `delete:packages`, `audit_log`).

Go 1.27.0 was installed on ENG-1 on 2026-09-05, so the toolchain claim is now measured on the machine the work actually happens on: `gofmt` clean, `go vet` exit 0, `go test ./...` green, and all six release targets building — linux/amd64, linux/arm64, linux/armv7, darwin/amd64, darwin/arm64, windows/amd64.

It is a **per-user** install at `%LOCALAPPDATA%\Programs\go`, from the official zip with its SHA-256 verified against `go.dev/dl/?mode=json` before extraction, with `go\bin` and `%USERPROFILE%\go\bin` added to the user `Path`. Per-user rather than machine-wide on purpose: the MSI needs elevation, and an unattended `msiexec /qn` from a non-elevated session fails *silently* — the same failure mode that produced the Bambu Studio update loop. Nothing about this install needs elevation, and `GOPATH` / `GOCACHE` land in the user profile rather than in the projects root — which mattered when that root was a Syncthing folder, and now simply keeps a multi-gigabyte build cache out of a directory full of repos. Promote it to a machine-wide install later if you want; nothing depends on where it lives.

mingw-w64 followed, for the race detector. `go test -race` needs cgo, and cgo needs a C compiler; without one ENG-1 reported `CGO_ENABLED=0` and `-race` failed outright with "requires cgo" rather than passing quietly. WinLibs GCC **16.2.0** (UCRT, POSIX threads, SEH) is now installed the same way — per-user at `%LOCALAPPDATA%\Programs\mingw64`, SHA-256 verified against the publisher's own `.sha256` before extraction, `bin` on the user `Path`. `go env CGO_ENABLED` now reports 1 and the suite is race-clean in about 16 seconds on the first run.

The extraction ran through a **scheduled task** rather than `Start-Process`, because a long child process started from a Cowork session dies with the process tree when a bridge call times out — that is what silently cancelled an earlier `winget` attempt mid-download. A scheduled task is detached from that tree and survives.

And the detector was proved able to go **red** before its green was believed: a throwaway module with eight goroutines incrementing a shared int returned `WARNING: DATA RACE` and exit 1.

Sequencing note

Phases 1 and 2 are the whole of the "#1 priority". Nothing in phases 3–5 should start before a stock YARG client is reading songs from a self-hosted server, because every later decision depends on what that turns out to actually require.